

GLOBO2-PE · DIRETRIZES DE ENGENHARIA

Como o projeto vai caminhar a partir de agora

Commits semânticos, refatoração contínua com Tidy First, squads com sublíderes e uma esteira ágil que transforma prioridade em entrega.

Conventional Commits

Tidy First (Kent Beck)

Squads & Sublíderanças

Esteira Ágil (Kanban)

Projeto: Media Compose Dashboard (globo2-PE) · Preparado por Lucas Alecrim · BSACorp
· Agosto 2026

1 Objetivo deste documento

O globo2-PE cresceu de um protótipo para um ecossistema com agente de coleta, backend, worker ETL, serviço de IA e dashboard. Esse crescimento exige regras claras: como escrevemos commits, como refatoramos sem quebrar comportamento, quem é responsável por cada parte do sistema e como o trabalho flui do backlog até produção. Este documento define essas quatro frentes e passa a valer para todos os subgrupos a partir desta apresentação.

O sistema hoje é composto por seis camadas principais, refletidas na estrutura de pastas do repositório: `agent/` (coleta via Watchdog), `backend/` (API Flask + Worker ETL), `data_ia/` (predição com Scikit-Learn), `frontend/` (dashboard Next.js), além da infraestrutura de `postgres/`, `redis_config/` e orquestração via `docker-compose.yml`. As regras abaixo foram desenhadas em cima dessa arquitetura real, não de forma genérica.

A partir de agora, todo Pull Request precisa: (1) seguir o padrão de commit semântico, (2) separar mudança estrutural de mudança de comportamento seguindo o Tidy First, (3) estar associado a um subgrupo responsável e (4) percorrer a esteira ágil descrita na seção 4.

2 Padrão de commits semânticos

Usaremos o padrão **Conventional Commits**: `tipo(escopo): descrição no imperativo`. O **escopo** deve ser, sempre que possível, o nome do subgrupo/módulo afetado — `backend`, `frontend`, `ia`, `agent`, `devops`, `docs`, `qa` — o que já conecta o commit ao dono da mudança.

Tipo	Quando usar	Exemplo real no projeto
feat	Nova funcionalidade ou comportamento visível para o usuário/sistema	<code>feat(backend): adiciona endpoint GET /dashboard/historico</code>
fix	Correção de bug/comportamento incorreto	<code>fix(ia): corrige fallback quando editor tem menos de 10 amostras</code>
refactor	Mudança estrutural sem alterar comportamento (ver Tidy First, seção 3)	<code>refactor(agent): extrai resolver_usuario para função pura</code>
chore	Tarefas de manutenção que não afetam código de produção (deps, configs, scripts)	<code>chore(devops): atualiza versão do redis no docker-compose</code>
docs	Documentação apenas (README, comentários, diagramas)	<code>docs(readme): documenta rota POST /retrain</code>
test	Criação ou ajuste de testes, sem tocar em código de produção	<code>test(backend): cobre event_routes com fila cheia</code>
style	Formatação, lint, espaçamento — zero impacto em lógica	<code>style(frontend): aplica prettier nos componentes do dashboard</code>
perf	Melhoria de performance mensurável	<code>perf(data_ia): reduz tempo de retrain com cache de dataframe</code>
build	Mudanças no processo de build/dependências	<code>build(frontend): migra para Next.js 15</code>
ci	Pipelines de integração contínua	<code>ci: adiciona lint obrigatório no pull request</code>
revert	Reversão de um commit anterior	<code>revert: reverte feat(backend) endpoint instável</code>

```
feat(data_ia): adiciona previsão por ilha além de por editor # título curto,
imperativo, até ~72 caracteres
```

```
Treina um modelo secundário agrupando por ilha quando o editor
não tem histórico suficiente (menos de 10 edições concluídas).
```

```
# corpo explica o "porquê", não repete o título
Refs: #128
```

Regra de ouro: um commit muda um tipo de coisa. Nunca misture `refactor` com `feat` no mesmo commit — se você reorganizou o código e adicionou uma funcionalidade, isso são no mínimo dois commits (e

idealmente dois momentos do PR). Essa é exatamente a disciplina do Tidy First, detalhada a seguir.

3 Refatoração contínua com Tidy First

Baseado em *"Tidy First? A Personal Exercise in Empirical Software Design"* (Kent Beck, O'Reilly). A ideia central: separe sempre **mudanças estruturais** (como o código está organizado) de **mudanças de comportamento** (o que o código faz). Pequenas arrumações ("tidying") feitas *antes* de implementar uma feature tornam a mudança de comportamento mais simples, mais segura e mais fácil de revisar.

Por que "tidy first" e não "tidy depois"

Quando o código está bagunçado, a tentação é misturar arrumação com a feature nova em um único commit gigante — difícil de revisar e difícil de reverter se algo quebrar. A regra prática: **antes** de tocar na lógica de negócio, faça as pequenas arrumações que tornam a mudança óbvia. Cada tidying vira um commit `refactor` isolado, sem alterar nenhum teste; só depois vem o commit `feat / fix` com a mudança de comportamento em cima do código já limpo.

Mini-refatorações (tidyings) recomendadas no dia a dia

Tidying	O que é	Exemplo no globo2-PE
Guard Clauses	Trocar ifs aninhados por retornos antecipados	Validar payload de <code>POST /events</code> com early return em vez de aninhar o processamento
Dead Code	Remover código não usado	Remover rotas de debug esquecidas em <code>event_routes.py</code>
Normalize Symmetries	Deixar código parecido de fato parecido (mesmo estilo para o mesmo problema)	Unificar como cada rota do Flask trata erro de conexão com Redis
Explaining Variables / Constants	Nomear expressões complexas e números mágicos	Extrair <code>MINIMO_AMOSTRAS_TREINO = 10</code> em vez do literal <code>10</code> no <code>predictor.py</code>
Explicit Parameters	Tornar dependências implícitas em parâmetros explícitos	Passar a conexão Redis para <code>worker.py</code> em vez de acessá-la como global
Chunk Statements	Agrupar blocos relacionados com uma linha em branco + comentário curto	Separar "parse do evento" / "gravação no Redis" / "chamada à IA" dentro de <code>event_routes.py</code>
Extract Helper	Extrair um trecho repetido para uma função nomeada	Extrair o parser <code><PROJETO> <CODIGO> <ILHA></code> do nome de pasta para uma função testável isolada
Reading / Cohesion Order	Reordenar funções/campos para refletir a ordem em que são lidos ou usados juntos	Reordenar rotas em <code>dashboard_routes.py</code> na ordem em que o frontend as consome

New Interface, Old Implementation

Criar uma interface melhor que chama a implementação antiga por dentro, migrando aos poucos

Criar `PredictorClient.prever(editor, tamanho_mb)` encapsulando a chamada HTTP crua a `data_ia`

Fluxo recomendado por Pull Request

1. Identifique a feature/fix a implementar.
2. Se o código está confuso onde você vai mexer, aplique 1–3 tidying primeiro, cada um em commit `refactor` próprio.
3. Rode os testes após cada tidying — comportamento não deve mudar.
4. Implemente a mudança de comportamento em commit(s) `feat / fix` separados.
5. Abra o PR com os commits visíveis nessa ordem — o revisor primeiro vê a limpeza, depois a mudança real.

Quando NÃO fazer tidy first

- Se a arrumação é grande o suficiente para virar um épico próprio (ex.: reescrever a camada de acesso ao Postgres) — trate como projeto à parte, com o DevOps/Backend lead avaliando o custo.
- Se não há testes cobrindo a área — escreva um teste de caracterização antes de tidyar.
- Perto de um deploy crítico ou janela de gravação ao vivo — tidying esperam, estabilidade não.

Definition of Done para refactor: todos os testes existentes continuam verdes, nenhum arquivo de teste foi alterado, e o diff não introduz nenhuma mudança de output observável.

4 Subgrupos e sublideranças

Cada subgrupo tem um sublíder responsável por revisar PRs do seu escopo, garantir aderência às regras de commit/Tidy First e representar a área na cerimônia de planning. Abaixo, a divisão mapeada na arquitetura real do repositório:

Backend

Sublíder Backend

- API Flask (`backend/app.py` , `routes/`)
- Worker ETL (`backend/worker/worker.py`)
- Integração com Redis e chamada ao serviço de IA

Escopo de commit: `backend` , `worker`

Frontend

Sublíder Frontend

- Dashboard Next.js + Recharts (`frontend/`)
- Consumo de `GET /dashboard` , polling e estados de loading/erro
- Consistência visual do painel gerencial

Escopo de commit: `frontend`

Design / UX

Sublíder Design

- Padrões visuais do dashboard e componentes reutilizáveis
- Fluxos de leitura para o gestor (status, previsão de término)
- Acessibilidade e consistência de cores/tipografia

Escopo de commit: `ui` , `design`

QA

Sublíder QA

- Testes de integração da esteira: Watchdog → Flask → Redis → Worker → Postgres → IA → Dashboard
- Cenários de borda (fila cheia, IA fora do ar, pasta mal nomeada)
- Gate de qualidade antes de qualquer merge para produção

Escopo de commit: `test` , `qa`

Dados

Sublíder Dados

- Modelagem das tabelas `usuarios` , `projetos` , `eventos` , `edicoes`
- Qualidade e consistência dos dados persistidos pelo Worker
- Métricas agregadas ainda pendentes no `/dashboard` (tempo médio, concluídos hoje)

Escopo de commit: `data` , `db`

IA

Sublíder IA

- Modelo de regressão linear de ETC (`data_ia/predictor.py`)
- Estratégia de fallback (por editor → global → heurística de 1.8 min/100MB)
- Ciclo de retreinamento e monitoramento de precisão do modelo

Escopo de commit: `ia` , `data_ia`

DevOps

Sublíder DevOps

- `docker-compose.yml` , `start.sh` / `stop.sh` , orquestração de containers

Documentação

Sublíder Docs

- README, diagramas de arquitetura, guias de onboarding

- Configuração de Redis (`redis_config/`) e variáveis de ambiente
- CI/CD, observabilidade e saúde dos serviços (`/health`)

Escopo de commit: `devops` , `ci` , `build`

- Documentação das rotas de API e variáveis de ambiente
- Manutenção deste próprio documento de diretrizes

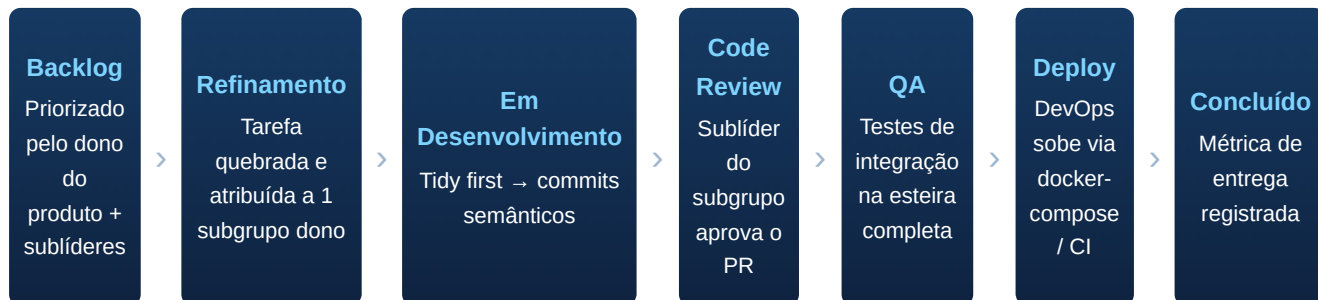
Escopo de commit: `docs`

Nota sobre o Agente de Coleta: `agent/monitor.py` e `usuario_map.py` ficam sob responsabilidade compartilhada de Backend + DevOps, já que envolvem tanto lógica de negócio (parser de pastas) quanto operação em estações de trabalho reais (deploy do Watchdog).

Metodologia ágil e esteira funcional

Adotamos um modelo híbrido Scrum/Kanban: sprints de **2 semanas** para dar ritmo de planejamento e revisão, com um **quadro Kanban contínuo** para visualizar o fluxo real de cada tarefa entre os subgrupos.

Esteira de trabalho (pipeline)



Cerimônias

Cerimônia	Frequência	Participantes	Saída
Sprint Planning	A cada 2 semanas	Todos os sublíderes	Tarefas do backlog atribuídas a subgrupos com estimativa
Daily (assíncrona ou 15min)	Diária	Cada subgrupo internamente	Bloqueios visíveis no quadro
Review de Sprint	A cada 2 semanas	Todos + stakeholders	Demonstração do incremento (ex.: nova predição no dashboard)
Retrospectiva	A cada 2 semanas	Todos os sublíderes	Ajustes no processo, inclusive nestas regras
Sync entre subgrupos	Semanal	Sublíderes com dependência direta (ex.: Backend ↔ IA ↔ Dados)	Alinhamento de contratos de API/esquema de dados

Como uma tarefa é atribuída a um subgrupo

- Toda issue nova recebe uma **label de subgrupo** (`backend` , `frontend` , `ia` , `dados` , `qa` , `devops` , `docs` , `design`) já no refinamento.
- Issues que cruzam mais de um subgrupo (ex.: nova feature no dashboard que exige campo novo na API) recebem múltiplas labels e são quebradas em subtarefas, uma por subgrupo, linkadas entre si.
- O sublíder do subgrupo é o responsável por puxar a tarefa para "Em Desenvolvimento" e garantir que ela segua as regras de commit e Tidy First.
- Nenhuma tarefa avança de QA para Deploy sem aprovação do sublíder de QA.

Definition of Done (esteira completa)

- ✓ Commits seguem o padrão semântico com escopo do subgrupo correto.

- ✓ Mudanças estruturais e de comportamento estão em commits separados (Tidy First).
- ✓ PR revisado e aprovado pelo sublíder responsável.
- ✓ Testes automatizados cobrindo o novo comportamento, validados por QA.
- ✓ Documentação (README/rotas/variáveis) atualizada quando aplicável.
- ✓ Deploy validado em ambiente local via `docker-compose` antes de ir para produção.

A partir desta apresentação, todo o time trabalha sobre quatro pilares: commits semânticos com escopo por subgrupo, refatoração disciplinada via Tidy First, subgrupos com sublideranças claras mapeadas na arquitetura real do sistema, e uma esteira ágil que leva cada tarefa do backlog à produção com donos e critérios de qualidade definidos em cada etapa.

Pilar	Regra central
Commits	<code>tipo(escopo): descrição</code> — nunca misturar tipos no mesmo commit
Refatoração	Tidy first, depois behavior change — sempre em commits/PRs separados
Subgrupos	8 squads: Backend, Frontend, Design/UX, QA, Dados, IA, DevOps, Documentação
Esteira	Backlog → Refinamento → Dev → Code Review → QA → Deploy → Concluído

Documento vivo — atualizações e dúvidas devem ser tratadas como issue no repositório, escopo `docs`, e revisadas na retrospectiva.